

ROYAL UNIVERSITY OF PHNOM PENH

Faculty of Computer Science and Engineering

HOMEWORK 1

Interfaces, Polymorphism and Virtual Methods

Subject	OOAD & Programming
Student Name	Mao Visal
Class	M4
Teacher	Dan Vournpheng
Date	September 2026

Table of Contents · មាតិកា

1. Interfaces
2. Polymorphism
3. Virtual Methods
4. Mini Project — Animal Management System
5. Conclusion
6. References

INTRODUCTION

Introduction · សេចក្តីផ្តើម

OOP (Object-Oriented Programming) គឺជាទម្រង់នៃការសរសេរកូដដែលផ្តោតលើការរៀបចំកម្មវិធីជាទម្រង់ Objects ដែលផ្តុំកន្លែងទិន្នន័យ (Properties) និងមុខងារ (Methods) ធ្វើការរួមគ្នា។ វាមានសារៈសំខាន់ខ្លាំងណាស់ ព្រោះវាជួយឱ្យកូដមានសណ្តាប់ធ្នាប់ ងាយស្រួលក្នុងការថែទាំ កាត់បន្ថយការសរសេរកូដដដែលៗ (Reusability) និងងាយស្រួលក្នុងការអភិវឌ្ឍកម្មវិធីខ្នាតធំ។

នៅក្នុង Homework នេះ ខ្ញុំនឹងធ្វើការសិក្សា និងបង្ហាញការយល់ដឹងលម្អិតអំពីគោលគំនិតសំខាន់ៗចំនួន ៣ របស់ OOP នៅក្នុងភាសា C# គឺ៖ Interface, Polymorphism និង Virtual Methods
ព្រមទាំងមានការអនុវត្តបង្កើតជា Mini Project ជាក់ស្តែងផងដែរ។

01 · INTERFACE

1. Interfaces

What? — តើ Interface ជាអ្វី?

Interface គឺប្រៀបបានទៅនឹង "កិច្ចសន្យា" (Contract) ឬ ប្លង់មេ នៅក្នុងការសរសេរកូដ។
វាមិនមានផ្នែកនូវកូដដែលដំណើរការជាក់ស្តែងនោះទេ (No implementation body) វាគ្រាន់តែប្រកាសប្រាប់ថា "ប្រសិនបើ Class ណាមួយចង់យក Interface នេះទៅប្រើ Class នោះ ត្រូវតែ មាន Method ទាំងនេះ"។

ចំណាំ៖ ឈ្មោះ Interface នៅក្នុង C# គេតែងតែដាក់អក្សរ I នៅខាងមុខជានិច្ច ឧទាហរណ៍ IAnimal ជាដើម។

Why? — ហេតុអ្វីយើងប្រើ Interface?

• Standardization (បង្កើតស្តង់ដារួម)៖ វាជួយធានាថា Class ផ្សេងៗគ្នាមាន Method ដូចគ្នា ដែលងាយស្រួលក្នុងការគ្រប់គ្រង។

• Multiple Inheritance៖ C# មិនអនុញ្ញាតឱ្យ Class មួយទទួលមរតកពី Class ច្រើនក្នុងពេលតែមួយទេ ប៉ុន្តែ Class មួយអាចយក Interface ច្រើនមកប្រើ (Implement) បាន។

When? — តើស្ថានភាពណាដែលគួរប្រើ Interface?

យើងប្រើវានៅពេលដែលយើងចង់ឱ្យក្រុមនៃ Class ផ្សេងៗគ្នា (ឧទាហរណ៍ Class Dog និង Cat) មានសកម្មភាពដូចគ្នា (ឧទាហរណ៍ សកម្មភាពបន្លឺសំឡេង MakeSound)
ទោះបីជារបៀបដែលពួកវាបញ្ចេញសកម្មភាពនោះខុសគ្នាក៏ដោយ (ឆ្កែព្រុស ឆ្មាស្រែកម៉ែវ)។

How? (Syntax)

SYNTAX

```
interface IExample
{
    void DoSomething(); // no body, ends with ;
}
```

Code Example

C# · INTERFACE

```
using System;

namespace OOP_Homework_1
{
    // 1. Interface IAnimal
    interface IAnimal
    {
        void MakeSound();
    }

    // 2. Dog implements IAnimal
    class Dog : IAnimal
    {
        public void MakeSound()
        {
            Console.WriteLine("Dog says: Woof Woof!");
```

```
        }
    }

    // 3. Cat implements IAnimal
    class Cat : IAnimal
    {
        public void MakeSound()
        {
            Console.WriteLine("Cat says: Meow Meow!");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            IAnimal myDog = new Dog();
            IAnimal myCat = new Cat();
            myDog.MakeSound();
            myCat.MakeSound();
        }
    }
}
```

🔑 Code Explanation

interface IAnimal: យើងបង្កើត Interface មួយដែលមាន Method ឈ្មោះ MakeSound()។

class Dog : IAnimal: សញ្ញា (:) មានន័យថា Class Dog យល់ព្រមចុះកិច្ចសន្យាយក IAnimal មកប្រើ (Implement)។ ដូច្នេះវា ដាច់ខាត ត្រូវតែសរសេរកូដលម្អិត (Body) ឱ្យ MakeSound()។

IAnimal myDog = new Dog();: នេះជាការបង្កើត Object (Object Creation) ដោយប្រើ Interface ជាប្រភេទ (Type) ដើម្បីរក្សាទុកកូនឆ្កែ (Dog instance)។

EXPECTED RESULT

```
--- Interface Example ---
Dog says: Woof Woof!
Cat says: Meow Meow!
```

02 · POLYMORPHISM

2. Polymorphism

What? — តើ Polymorphism ជាអ្វី?

Polymorphism (ពហុទម្រង់) គឺមកពីពាក្យក្រិក "Poly" មានន័យថា ច្រើន (Many) និង "Morph" មានន័យថា ទម្រង់ (Forms)។ សរុបមកវាមានន័យថា "ការមានទម្រង់ច្រើន"។ នៅក្នុង C# វាគឺជាសមត្ថភាពដែលអនុញ្ញាតឱ្យ Object ផ្សេងៗគ្នា (ឧទាហរណ៍ Dog និង Cat) អាចត្រូវបានគេចាត់ទុកជា ប្រភេទតែមួយ (ឧទាហរណ៍ IAnimal) ប៉ុន្តែពេលយើងបញ្ជាពួកវា ពួកវាបញ្ចេញសកម្មភាព (Behavior) ខុសៗគ្នាពីរៀងៗខ្លួន។

Why? — ហេតុអ្វីយើងប្រើវា?

- ភាពបត់បែន (Flexibility)៖ យើងអាចសរសេរកូដតែមួយកន្លែង ដែលអាចដំណើរការជាមួយ Object ជាច្រើនប្រភេទ។
- ងាយស្រួលថែទាំ (Maintainability)៖ ពេលយើងចង់បន្ថែមសត្វថ្មី (ឧទាហរណ៍ Bird) យើងមិនចាំបាច់កែកូដចាស់ច្រើនទេ គឺគ្រាន់តែបង្កើត Class ថ្មីជាការស្រេច។

How? — ២ ប្រភេទសំខាន់ៗ

Compile-time Polymorphism (Static)៖ ការប្រើឈ្មោះ Method ដូចគ្នា តែទទួល Parameter ខុសគ្នា (Method Overloading)។

Runtime Polymorphism (Dynamic)៖ ផ្តោតលើការហៅ Method ដែលត្រូវបាន implement ពី Interface ឬ Override ពី Base Class (ផ្តោតលើមួយនេះ ព្រោះវាត្រូវគ្នានឹង Homework)។

Code Example

C# · POLYMORPHISM

```
using System;
using System.Collections.Generic;

namespace OOP_Homework_1
{
    interface IAnimal { void MakeSound(); }

    class Dog : IAnimal
    {
        public void MakeSound()
        {
            Console.WriteLine("Dog says: Woof Woof!");
        }
    }

    class Cat : IAnimal
    {
        public void MakeSound()
        {
            Console.WriteLine("Cat says: Meow Meow!");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            List<IAnimal> myAnimals = new List<IAnimal>();
```

```
            myAnimals.Add(new Dog());
            myAnimals.Add(new Cat());

            foreach (IAnimal animal in myAnimals)
            {
                animal.MakeSound(); // one call, different results
            }
        }
    }
}
```

EXPECTED RESULT

```
--- Polymorphism Example ---
Dog says: Woof Woof!
Cat says: Meow Meow!
```

03 · VIRTUAL METHODS

3. Virtual Methods

What? — តើ Virtual Method ជាអ្វី?

Virtual Method គឺជា Method មួយនៅក្នុង Base Class ដែលបានសរសេរកូដ (Body) រួចរាល់ហើយ ប៉ុន្តែវាអនុញ្ញាតឱ្យ Child Class ដែលទទួលមរតក (Inheritance) ពីវា អាចទាញយក Method នោះមកកែប្រែកូដខាងក្នុងថ្មីបាន។

ពាក្យគន្លឹះសំខាន់ៗ (Keywords):

- virtual៖ ប្រើនៅក្នុងកូដរបស់ មេ ដើម្បីប្រាប់ថា "កូនៗអាចកែប្រែ Method នេះបានតាមចិត្ត!"។
- override៖ ប្រើនៅក្នុងកូដរបស់ កូន ដើម្បីប្រាប់ថា "ខ្ញុំសុំសរសេរជាន់ពីលើ Method របស់មេហើយ!" (ការធ្វើបែបនេះហៅថា Method Overriding)។
- Inheritance៖ ការទទួលមរតកពី Base class (មេ) ទៅ Derived class (កូន)។

Why? — ហេតុអ្វីយើងប្រើ Virtual Method?

វាជួយឱ្យយើងអាចបង្កើត សកម្មភាពលំនាំដើម (Default Behavior) នៅក្នុង Base Class។ ប្រសិនបើកូនមិនចង់កែប្រែទេ វានឹងប្រើប្រាស់សកម្មភាពរបស់មេដោយស្វ័យប្រវត្តិ។ តែបើកូនចង់មានលក្ខណៈពិសេសរៀងខ្លួន ទើបកូនចាំបាច់ប្រើ override កែប្រែវា។ នេះជាទម្រង់មួយទៀតនៃ Runtime Polymorphism។

When? — ពេលណាគួរប្រើ Virtual Method?

យើងប្រើវានៅពេលដែលមានទំនាក់ទំនងជា មេ-កូន (Inheritance) ហើយមេមានសកម្មភាពទូទៅមួយ តែយើងដឹងថាកូនខ្លះអាចនឹងចង់ធ្វើសកម្មភាពនោះខុសពីមេ។

Code Example

C# · VIRTUAL / OVERRIDE

```
using System;
using System.Collections.Generic;

namespace OOP_Homework_1
{
    // 1. Base Class
    class Animal
    {
        public virtual void MakeSound()
        {
            Console.WriteLine("Animal makes a general sound.");
        }
    }

    // 2. Child Class (overrides)
    class Dog : Animal
    {
        public override void MakeSound()
        {
            Console.WriteLine("Dog barks: Woof Woof!");
        }
    }

    // 3. Child Class (does not override)
    class Fish : Animal { }

    class Program
```

```
    {
        static void Main(string[] args)
        {
            Animal genericAnimal = new Animal();
            Animal myDog = new Dog();
            Animal myFish = new Fish();

            genericAnimal.MakeSound(); // parent
            myDog.MakeSound();         // overridden
            myFish.MakeSound();        // inherited
        }
    }
}
```

EXPECTED RESULT

```
--- Virtual Method Example ---
Animal makes a general sound.
Dog barks: Woof Woof!
Animal makes a general sound.
```

Important Comparison · ការប្រៀបធៀបសំខាន់ៗ

Concept · គំនិត	Meaning · អត្ថន័យ	Keyword · ពាក្យគន្លឹះ	Purpose · គោលបំណង
Interface	ប្លង់មេ ឬកិច្ចសន្យា ដែលមិនមានផ្នែកកូដលម្អិត។	`interface`	បង្កើតស្តង់ដារួម (Standardization) ដើម្បីឱ្យ Class ផ្សេងៗយកទៅ implement។
Polymorphism	ពហុទម្រង់ (សមត្ថភាពមានរូបរាងច្រើន)។	`virtual, override, interface`	អនុញ្ញាតឱ្យ Object ផ្សេងៗគ្នា (ដូចជាឆ្កែ និងឆ្មា) បញ្ចេញសកម្មភាពខុសៗគ្នា ពេលហៅឈ្មោះ Method តែមួយ។
Virtual Method	Method របស់ថ្នាក់មេ (Base Class) ដែលអនុញ្ញាតឱ្យថ្នាក់កូន (Child Class) កែប្រែបាន។	`virtual` (មេ) `override` (កូន)	ផ្តល់សកម្មភាពលំនាំដើមដល់ថ្នាក់កូន និងអនុញ្ញាតឱ្យថ្នាក់កូនកែប្រែ (Method overriding) ប្រសិនបើចាំបាច់។

04 · MINI PROJECT

Final Mini Project — Animal Management System

Goal

បង្កើតកម្មវិធីគ្រប់គ្រងសត្វ (Animal Management System) មួយដែលប្រើប្រាស់ Interface, Inheritance, Polymorphism, និង Virtual Method/Override រួមគ្នា។

C# · ANIMAL MANAGEMENT SYSTEM

```
using System;
using System.Collections.Generic;

namespace OOP_Homework_1
{
    // 1. Interface to define the standard
    interface IAnimal
    {
        void DisplayInfo();
    }

    // 2. Base Class implementing the interface
    class Animal : IAnimal
    {
        public string Name { get; set; }

        public Animal(string name) { Name = name; }

        public void DisplayInfo()
        {
            Console.WriteLine($"\n[Animal Info] Name: {Name}");
        }

        public virtual void MakeSound()
        {
            Console.WriteLine("Animal makes a generic sound.");
        }
    }

    // 3-5. Child Classes (Dog, Cat, Bird) — same pattern
    class Dog : Animal
    {
        public Dog(string name) : base(name) { }
        public override void MakeSound()
        {
            Console.WriteLine($"{Name} (Dog) barks: Woof! Woof!");
        }
    }

    class Cat : Animal
    {
        public Cat(string name) : base(name) { }
        public override void MakeSound()
        {
            Console.WriteLine($"{Name} (Cat) meows: Meow! Meow!");
        }
    }

    class Bird : Animal
```

```
    {
        public Bird(string name) : base(name) { }
        public override void MakeSound()
        {
            Console.WriteLine($"{Name} (Bird) chirps: Tweet! Tweet!");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Animal Management System ===");

            List<Animal> zoo = new List<Animal>();
            zoo.Add(new Dog("Kiki"));
            zoo.Add(new Cat("Mimi"));
            zoo.Add(new Bird("Piki"));

            foreach (Animal animal in zoo)
            {
                animal.DisplayInfo();
                animal.MakeSound();
            }
        }
    }
}
```

EXPECTED RESULT

```
=== Animal Management System ===

[Animal Info] Name: Kiki
Kiki (Dog) barks: Woof! Woof!

[Animal Info] Name: Mimi
Mimi (Cat) meows: Meow! Meow!

[Animal Info] Name: Piki
Piki (Bird) chirps: Tweet! Tweet!
```

CONCLUSION

Conclusion · សេចក្តីសន្និដ្ឋាន

ជាការសន្និដ្ឋាន តាមរយៈការធ្វើ Homework នេះ ខ្ញុំទទួលបានការយល់ដឹងច្បាស់អំពី៖

Interface :
របៀបបង្កើតកិច្ចសន្យា ឬបង់មេ ដើម្បីកំណត់ស្តង់ដារដល់ Class ផ្សេងៗ។

Virtual Method & Override :
របៀបប្រើប្រាស់ការទទួលមរតក (Inheritance) ដើម្បីផ្តល់សក្ដានុពលនាំដើមពីថ្នាក់មេ
និងការអនុញ្ញាតឱ្យថ្នាក់កូន (Child class) អាចសរសេរជាន់ពីលើ (Override) ដើម្បីកែប្រែសកម្មភាពនោះ។

Polymorphism :
ការប្រើប្រាស់ Interface ឬ Base Class ជា Reference ដើម្បីអាចបញ្ជា Object ច្រើនប្រភេទផ្សេងគ្នា
ឱ្យបញ្ចេញសកម្មភាពខុសៗគ្នាក្នុងពេលតែមួយដោយស្វ័យប្រវត្តិ។

REFERENCES

References · ឯកសារយោង

Microsoft Learn: Interface in C#. Retrieved from: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/interface

Microsoft Learn: Polymorphism. Retrieved from: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism

Microsoft Learn: Virtual Modifiers. Retrieved from: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/virtual